Role-Based Doctor Appointment Booking and Management System Using Spring Boot and RESTful Architecture

M.S. Venugopala Rao[1],
Project Guide
Department of Computer Science and Engineering,
Bonam Venkata Chalamayya Engineering College,
Affiliated to JNTU Kakinada, Andhra Pradesh–533210, India,

D. Devi Sri Priya[2], Ch. Bhavya[3], K. Lohitha Lakshmi Durga[4], N. Durga Saraswathi[5], A. Kanaka Mahalakshimi[6]
[123456]Department of Computer Science and Engineering,
Bonam Venkata Chalamayya Engineering College,
Affiliated to JNTU Kakinada, Andhra Pradesh–533210, India,

Abstract—Traditional doctor appointment booking systems often rely on manual, counter-based scheduling or loosely integrated digital platforms, resulting in long waiting times, appointment conflicts, and poor patient experience. This paper presents a Role-Based Doctor Appointment Booking and Management System built with Java, Spring Boot, React, and MySQL following a layered RESTful architecture. Three roles are supported — Patient, Doctor, and Administrator
— each with dedicated dashboards and domain operations. Patients browse doctors by specialization, book time-slotted appointments, and track status in real time. Doctors approve or reject requests and manage availability. Administrators oversee users, doctor profiles, and system data. A three-tier design separates the React presentation layer, Spring Boot application tier, and MySQL data tier with ACID guarantees. Multi-stage server-side validation enforces doctor availability, slot conflict detection, duplicate booking prevention, and status transition integrity. Functional evaluation confirms correctness of all core workflows.

Index Terms—Doctor Appointment System, Spring Boot, RESTful API, Role-Based Access Control, Healthcare Management, MySQL, Layered Architecture

I. Introduction

Healthcare appointment management systems play a vital role in improving patient experience and operational efficiency in the medical sector. Traditional appointment booking methods — phone calls, physical walk-in registration, and paper-based records — suffer from significant limitations. These manual approaches lack centralized control and real-time availability updates, causing frequent conflicts when multiple patients attempt to book the same doctor at the same time slot. Although some digital healthcare platforms exist, many are loosely integrated, lack proper role-based authorization, or fail to enforce atomic appointment lifecycle management.

The absence of structured role separation means that patients, doctors, and administrative staff operate without clearly defined access boundaries, leading to security vulnerabilities, data inconsistencies, and poor accountability. Manual approval workflows impose administrative overhead and slow response times, degrading patient satisfaction.

This paper presents a Doctor Appointment Booking and Management System demonstrating modern software engineering practices using Spring Boot and RESTful APIs. Three user roles — Patient, Doctor, and Administrator — are supported with dedicated workflows. Patients search for available doctors by specialization, book appointments, and monitor status in real time. Doctors view their appointment queue, approve or reject requests, and update availability. Administrators manage doctor profiles, patient accounts, and system operations.

A. Contributions

The key contributions are: (1) layered architectural design using the Controller-Service-Repository pattern; (2) comprehensive server-side validation covering appointment creation, status transitions, and conflict detection; (3) role-based access control enforcing distinct capabilities for all three roles; (4) atomic appointment lifecycle management; (5) React-based responsive role-specific dashboards; and (6) a prototype validated through scenariobased API testing.

II. Literature Review

A. Early Web-Based Healthcare Solutions

Early academic efforts focused on replacing manual appointment scheduling with online forms and databasebacked storage. These demonstrated improved processing speed but suffered from procedural architectures mixing data access with business logic [1]. Security was limited to simple login forms without role differentiation.

B. Modern Framework Implementations

Recent studies adopted modern web frameworks demonstrating responsive frontend design and REST API integration for healthcare applications. However, many acknowledged limitations in transaction management and lacked proper role-based access control [5]. Kumar and Singh [6] highlighted the importance of RBAC in management systems. Verma et al. [7] noted that validation, conflict

prevention, and atomic operations remain recurring engineering challenges.

C. Identified Gaps

Analysis reveals recurring gaps: (1) absence of clear architectural separation; (2) validation enforced only at the UI level; (3) non-atomic appointment confirmation workflows; (4) insufficient overlapping slot detection; (5) limited administrative control; and (6) inconsistent appointment cancellation handling. The proposed system addresses all six gaps.

III. System Architecture

The proposed system adopts a three-tier client-server architecture separating presentation, application logic, and data persistence concerns, as shown in Fig. 1.

A. RESTful API Design

The application exposes REST endpoints across domainspecific controllers: UserController (registration/authentication), DoctorController (profile and availability), AppointmentController (booking lifecycle), and AdminController (system administration). Each controller follows REST resource naming conventions with appropriate HTTP verbs and JSON interchange format.

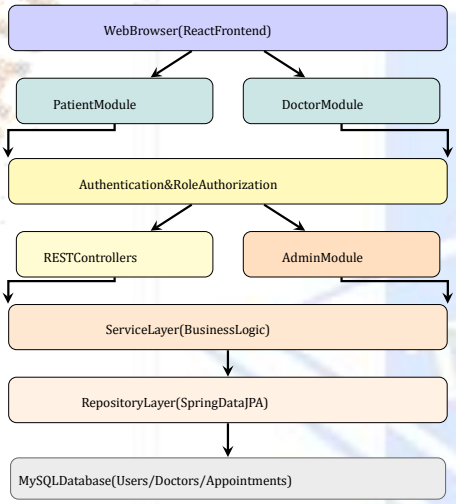


Figure 1: Three-tier system architecture

B. Three-Tier Architecture

Presentation Tier: React-based web interface provides doctor search, appointment booking, status tracking, and administrative operations via Axios HTTP calls.

Application Tier: Spring Boot 3.x encapsulates business logic, validation, and role-based authorization using the Controller-Service-Repository pattern with Spring Security.

Data Tier: MySQL managed through Hibernate ORM provides ACID-compliant persistent storage with referential integrity enforcement.

C. Data Model and Entity Relationships

The data model comprises four core JPA entities: User (credentials and role), Doctor (profile and availability), Appointment (reservation with status), and Payment (consultation payment tracking).

IV. System Design and Implementation

A. Authentication and Role Management

During registration the system validates email uniqueness before persisting a User with role (PATIENT, DOCTOR, or ADMIN). Passwords are stored as BCrypt hashes. The login endpoint validates credentials and returns role information, enabling the React frontend to redirect to the appropriate dashboard.

```
@Bean
public SecurityFilterChain filterChain(
    HttpSecurity http) throws Exception {
  http.csrf(c -> c.disable())
   .authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**")
        .permitAll()
    .requestMatchers("/api/admin/**")
        .hasRole("ADMIN")
    .requestMatchers("/api/doctor/**")
        .hasAnyRole("DOCTOR","AD
        MIN")
    .requestMatchers("/api/appointment
    /**")
        .hasAnyRole("PATIENT","DOC
        TOR","ADMIN")
    .anyRequest().authenticated());
  return http.build();
}
```

Listing 1: Spring Security role-based configuration

B. Doctor Management Module

Doctor registration records name, specialization, email, phone, and availability status (AVAILABLE/UNAVAILABLE). Patients search via GET /api/doctors?specialization={spec}. Administrators add, update, or remove doctor accounts secured behind ADMIN authorization.

C. Appointment Lifecycle and Validation

Booking submission initiates multi-stage server-side validation: (1) patient authentication; (2) doctor existence and availability; (3) future date-time requirement; (4) duplicate slot detection; and (5) status initialization to PENDING. Fig. 2 illustrates the complete workflow.

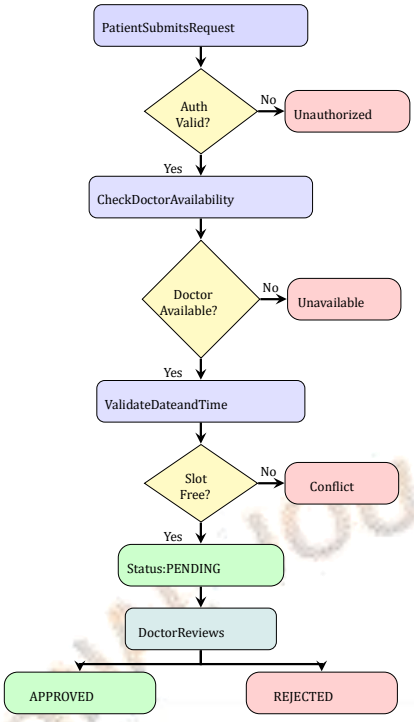


Figure 2: Appointment booking workflow with validation

```
@Transactional
public Appointment bookAppointment(
    Long userId, Long doctorId,
    LocalDate date, LocalTime time) {
  User user =
  userRepository.findById(userId)
    .orElseThrow(() -> new
      RuntimeException("User not
      found"));
  Doctor doctor =
  doctorRepository.findById(doctorId)
    .orElseThrow(() -> new
      RuntimeException("Doctor not
      found"));
  if
    (date.isBefore(LocalDate.now()
    ))          throw          new
    RuntimeException(
      "Date must be in the future");
  boolean conflict = appointmentRepository
    .existsByDoctorAndDateAndT
      ime( doctor, date, time);
  if        (conflict)        throw        new
    RuntimeException("Slot already
    booked");
  Appointment a = new Appointment();
a.setUser(user);a.setDoctor(doctor);
  a.setAppointmentDate(date);
  a.setAppointmentTime(time);
  a.setStatus("PENDING");
  return appointmentRepository.save(a);
}
```

Listing 2: Appointment service with validation

D. Appointment Cancellation Workflow

Cancellation executes within a @Transactional boundary: (1) retrieve the entity; (2) validate status is PENDING or APPROVED; (3) update to CANCELLED; (4) persist. Cancelling a REJECTED appointment throws InvalidStateException.

E. Administrative System Configuration

Administrators register new doctors, modify profiles, and deactivate accounts. The admin dashboard presents aggregated statistics including total, pending, approved, and rejected appointment counts, and provides complete appointment oversight across all doctor-patient pairs.

F. DTOs and Exception Handling

AppointmentRequestDTO encapsulates booking data. A centralized @RestControllerAdvice global exception handler translates domain exceptions to HTTP status codes and descriptive JSON error messages.

V. Entity Relationship Design

Fig. 3 presents the core entity relationships. Patient and Doctor entities are linked through the Appointment entity via many-to-one foreign keys.

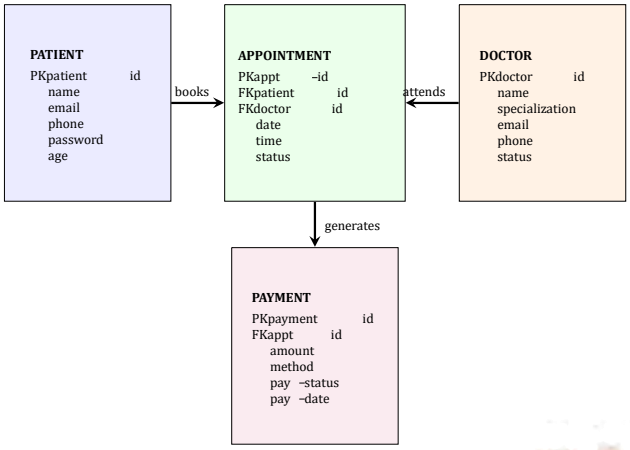


Figure 3: Core entity relationships

VI. Sequence Design

Fig. 4 traces the appointment booking interaction from the patient browser through the React frontend, Spring Boot backend, and MySQL database.

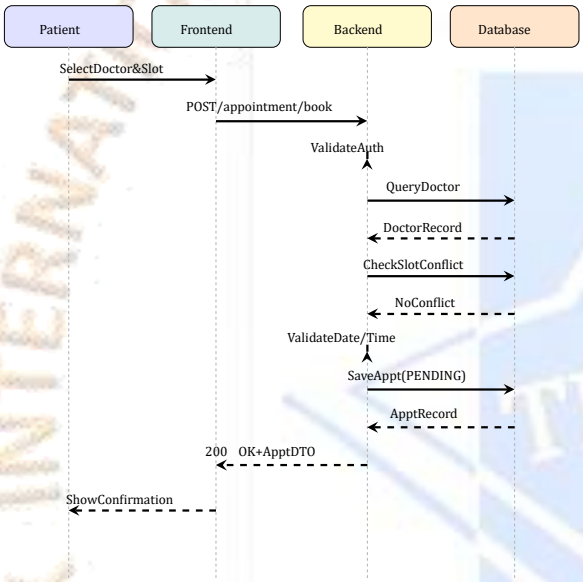


Figure 4: Sequence diagram: appointment booking operation

VII. Results and Performance Evaluation

A. Functional Validation Results

Functional validation used scenario-based REST API testing with Postman across all three user roles. Ten core validation rules were verified: (1) future date enforcement; (2) doctor existence; (3) doctor availability status; (4) duplicate slot detection; (5) patient authentication; (6) status transition enforcement; (7) role-based authorization; (8) appointment ownership; (9) information completeness; and (10) JSON response correctness. All ten demonstrated correct enforcement.

Transaction atomicity was confirmed through simulated failure testing, with complete rollback on booking and cancellation failures preventing partial state updates.

Table 1: Representative Test Case Results

Test Case	Expected	Result
Patient Registration	201 Created	PASS
Valid Login	200 + role info	PASS
Book Valid Appointment	200 + PEND-ING	PASS
Duplicate Slot Booking	400 Conflict	PASS
Past Date Booking	400 Validation Err	PASS
Doctor Approval	200 + AP-PROVED	PASS
Unauthorised Admin Access	403 Forbidden	PASS
Cancel PENDING Appt	200 + CAN-CELLED	PASS
Cancel REJECTED Appt	400 Invalid State	PASS
Admin Add Doctor	201 Created	PASS

B. System Performance Characteristics

Read Operations: Doctor search and appointment listing exhibit 100–200ms average latency due to indexed database access.

Write Operations: Transactional booking and cancellation incur 200–350ms overhead due to multi-stage validation and @Transactional coordinated writes.

Scalability: Spring Boot embedded Tomcat suits small to medium healthcare facilities. Docker/Kubernetes horizontal scaling is identified as a future enhancement.

VIII. Comparative Analysis

Table 2: Manual System vs. Proposed System

Feature	Manual	Proposed
Booking Time	Minutes	Seconds
Availability	Not real-time	Real-time
Conflict Check	Error-prone	Automated
Role Separation	None	Full RBAC
Data Security	Low	BCrypt+Auth
Scalability	Very limited	Spring Boot
Status Updates	Manual	Instant
Admin Dash-board	None	Centralized

IX. Discussion

A. Design Validation and Key Observations

The Controller-Service-Repository pattern provides clear layer responsibilities: REST controllers handle HTTP concerns; services encapsulate business rules and validation; repositories abstract database operations. Rolebased authorization correctly enforces all access boundaries. All ten domain validation rules behaved correctly during testing.

B. Implementation Challenges

Challenges encountered: (1) slot conflict detection under concurrent access required careful transaction isolation configuration; (2) maintaining clean layer separation during multi-stage validation; (3) managing JPA bidirectional

relationships to avoid N+1 query issues; and (4) configuring Spring Security CORS/CSRF policies for the React frontend.

C. Limitations and Constraints

Current limitations: (1) basic credential authentication without JWT; (2) payment processing not integrated with a real gateway; (3) no email/SMS notification services; (4) no recurring doctor availability scheduling; and (5) no load testing beyond single-instance usage.

X. Conclusion and Future Work

A. Conclusion

This paper presented a Role-Based Doctor Appointment Booking and Management System demonstrating modern software engineering practices with Spring Boot, React, and MySQL. The implementation addressed all six identified gaps: three-tier architectural separation, server-side validation, role-based access control, and atomic lifecycle management. Functional validation confirmed correctness of all ten core domain rules. The system delivers improved booking efficiency, real-time availability, conflict prevention, and centralized administrative oversight.

B. Future Work

Planned enhancements: (1) JWT-based stateless authentication; (2) payment gateway integration; (3) automated email/SMS notifications; (4) recurring doctor availability scheduling; (5) telemedicine video consultation integration; (6) AI-based doctor recommendation; (7) mobile applications for Android and iOS; (8) analytics dashboards; (9) Docker/Kubernetes deployment; and (10) comprehensive automated test suites.

Acknowledgment

The authors thank Bonam Venkata Chalamayya Engineering College and the Department of Computer Science and Engineering for providing laboratory facilities. Sincere gratitude is expressed to project guide Mr. M.S. Venugopala Rao, M.Tech (Ph.D), Associate Professor, for expert guidance throughout the academic year 2023–2026. The authors also thank Dr. Meerasharif Sheik, Professor and HOD, and Dr. Maheswar Dutta, Principal, for their continued support.

References

[1] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. New York: McGraw-Hill, 2015.

[2] H. Schildt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill, 2019.

[3] K. Sierra and B. Bates, *Head First Java*, 2nd ed. Sebastopol, CA: O'Reilly, 2018.

[4] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston: Pearson, 2016.

[5] C. Walls, *Spring Boot in Action*. Shelter Island, NY: Manning, 2016.

[6] S. Kumar and T. R. Singh, "Role-Based Access Control for Healthcare Management Systems," *Int. J. Computer Science and Information Technologies*, vol. 6, no. 4, pp. 3456–3461, 2015.

[7] P. Verma, R. Sharma, and A. Kumar, "A Survey on Online Doctor Appointment Reservation Systems," *Int. J. Advanced Research in Computer Science*, vol. 8, no. 7, pp. 2340–2345, 2017.

[8] M. Patel and N. Shah, "Design and Implementation of Web-Based Healthcare Appointment System Using Spring Framework," *Int. J. Engineering Research and Technology*, vol. 7, no. 3, pp. 215–220, 2018.

[9] A. Singh, K. Mishra, and R. Gupta, "Transaction Management in Online Booking Systems," *J. Database Management*, vol. 30, no. 2, pp. 45–62, 2019.

[10] MySQL Documentation. [Online]. Available: https://dev.mysql.com/doc/

[11] Spring Boot Official Documentation. [Online]. Available: https://spring.io/projects/ spring-boot

[12] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. California, Irvine, 2000.